PBA: 9. Regression and Model Fit

Fall 2025

Jaewon (“Jay-one”) Yoo
National Tsing Hua University

Roadmap

1. Prediction
2. Modeling With a Line
3. Linear Regression in *R*
4. Model Fit

1/ Prediction

Predicting Body Weight

- Predicting body weight with physical activity: `health` data
  - Real health data from an anonymous friend: `Jimmy Q. Boxplot`

Name	Description
`date`	date of measurements
`active_calories`	calories burned
`steps`	number of steps taken (in 1,000s)
`weight`	weight (kg)
`steps_lag`	steps on day before (in 1,000s)
`calories_lag`	calories burned on day before

Another DALLE3-Gen Image. Prompt: `Create a quality illustration of the bivariate relationship between the steps taken (walking) and body weight.`

Predicting Using Bivariate Relationship

- Goal: What's our best guess about Y_i if we know what X_i is?
  - What's our best guess about `Jimmy Boxplot`'s weight this morning if I know how many steps he took yesterday?
- Terminology:
  - **Dependent/outcome variable**: what we want to predict (weight).
  - **Independent/explanatory variable**: what we're using to predict (steps).

Weight Data

- Import the data:

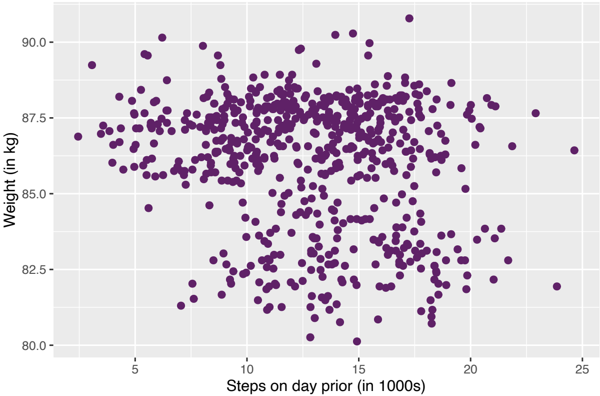
```
# Don't forget to load tidyverse and ggplot2!
> health <- as_tibble(read.csv(url("https://bit.ly/47uyilg"))); health

# A tibble: 644 × 6
   date       active_calories steps weight steps_lag calorie_lag
   <chr>                <dbl> <dbl>  <dbl>     <dbl>       <dbl>
 1 2015-08-09             480  17.5   86.2      NA            NA
 2 2015-08-10             996. 18.4   86.7      17.5         480
 3 2015-08-11            1127. 19.6   86.2      18.4         996.
 4 2015-08-12             522. 10.4   85.8      19.6        1127.
 5 2015-08-13             844. 18.7   86.3      10.4         522.
 6 2015-08-14             396.  9.14  86.1      18.7         844.
 7 2015-08-15             423.  8.69  85.4       9.14        396.
 8 2015-08-16             958. 13.8   86.4       8.69        423.
 9 2015-08-17             597. 11.9   86.7      13.8         958.
10 2015-08-18            1378. 24.6   86.8      11.9         597.
#  634 more rows
#  Use `print(n = ...)` to see more rows
```

- Drop NAs using drop_na():

```
> health <- drop_na(health)
```

Plotting Data: Weight and Steps

```
> Tsinghua_Purple <- "#5f2167"  # Hex code for Tsinghua Purple
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple, size = 2) +
    labs(
      x = "Steps on day prior (in 1000s)",
      y = "Weight (in kg)"
    )
```



Predicting One Variable with Another

- Prediction with access to just Y: average of the Y values.
- Prediction with another variable: for any value of X, what's the best guess about Y?
  - Need a function $y = f(x)$ that maps values of X into predictions.
  - **Machine learning**: fancy ways to determine $f(x)$.
- Example: What if `Jimmy` did 5,000 steps today ($x = 5k$)?
  $\rightsquigarrow$ What's our best guess about his weight tomorrow morning?

Start with Looking at a Narrow Strip of X

- Let's find all values that round to 5,000 steps:

```
> health |>
    filter(round(steps_lag) == 5)

# A tibble: 12 × 6
   date       active_calories steps weight steps_lag calorie_lag
   <chr>                <dbl> <dbl>  <dbl>     <dbl>       <dbl>
 1 2015-09-08           1111. 15.2    86.6      5.02        410.
 2 2015-12-12            728. 14.7    85.9      5.36        259.
 3 2015-12-28            430.  8.94   87.2      5.19        314
 4 2016-01-29            475.  8.26   87.7      4.95        314.
 5 2016-02-14            264.  5.42   88.1      4.86        297.
 6 2016-02-15            892. 13.1    87.7      5.42        264.
 7 2016-05-02            627. 11.8    87.2      5.04        283.
 8 2016-06-27            352.  7.21   86.5      4.93        212.
 9 2016-07-22            766. 14.8    85.9      4.96        251.
10 2016-11-25            452   9.4    88.4      5.26        295
11 2016-11-28            577. 11.8    87.6      4.97        304.
12 2016-12-30            621. 12.4    89.6      5.42        371.
```

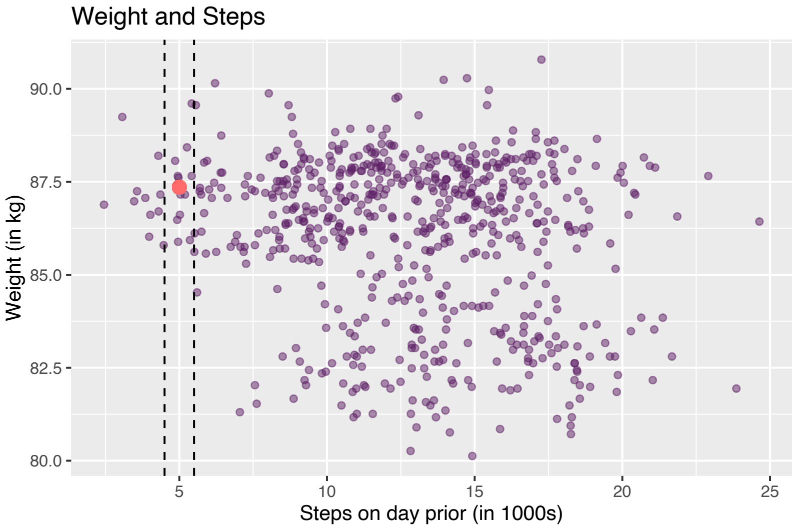
Best Guess About Y for $x = 5k$

- Best prediction about weight for a step count of roughly 5,000 is the average weight for observations around that value:

```
> mean_wt_5k_steps <- health |>
    filter(round(steps_lag) == 5) |>
    summarize(mean(weight)) |>
    pull() # indexing operator tailored to dplyr pipelines

> mean_wt_5k_steps
[1] 87.35256
```

- Plot the best guess:

```
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple, alpha = 0.5) +
    labs(
      x = "Steps on day prior (in 1000s)",
      y = "Weight (in kg)",
      title = "Weight and Steps"
    ) +
    geom_vline(xintercept = c(4.5, 5.5), linetype = "dashed") +
    geom_point(aes(x = 5, y = mean_wt_5k_steps), color = "indianred1", size = 3)
```



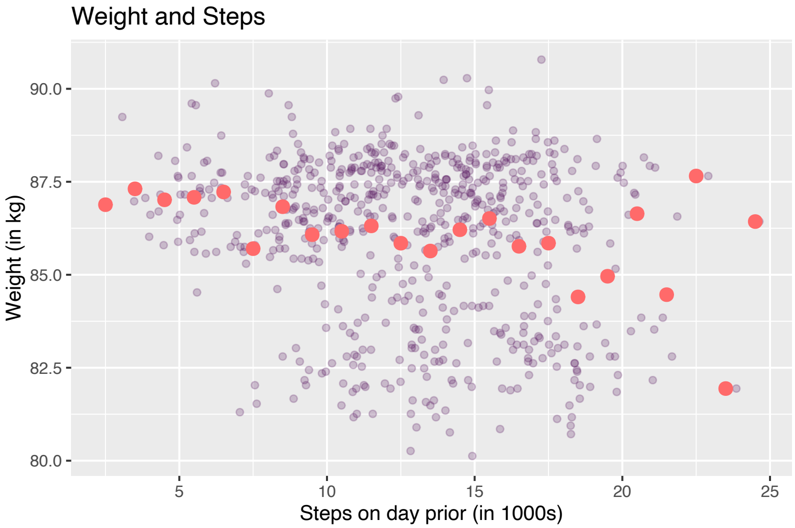
Weight and Steps
Weight (in kg)
90.0
87.5
85.0
82.5
80.0
5
10
15
20
25
Steps on day prior (in 1000s)

Binned Means

- Binning (or "bucketing") is the process of transforming continuous data into discrete bins or groups.
  - Discrete data like `steps on day prior` can be further discretized into larger bins.
  - Binned means for $x = 6k$? $x = 7k$? $x = 8k$?

- We can use a `stat_summary_bin()` to add these binned means all over the scatter plot:

```
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple, alpha = 0.25) +
    labs(
      x = "Steps on day prior (in 1000s)",
      y = "Weight (in kg)",
      title = "Weight and Steps"
    ) +
    stat_summary_bin(fun = "mean", color = "indianred1", size = 3,
                     geom = "point", binwidth = 1)
```

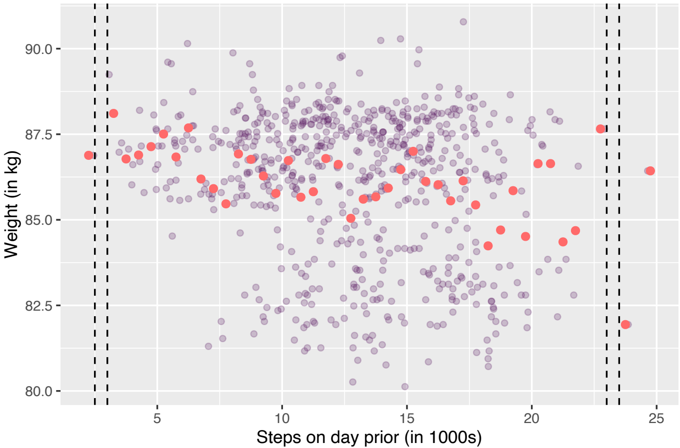

Weight and Steps
90.0
87.5
85.0
82.5
80.0
Weight (in kg)
5
10
15
20
25
Steps on day prior (in 1000s)

Smaller Means

- But, what happens when we make the bins too small?:

```
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple, alpha = 0.25) +
    labs(x = "Steps on day prior (in 1000s)", y = "Weight (in kg)") +
    stat_summary_bin(fun = "mean", color = "indianred1", size = 2,
                     geom = "point", binwidth = 0.5) +
    geom_vline(xintercept = c(2.5, 3, 23, 23.5), linetype = "dashed")
```

Gaps and **bumps**:



- More generally,
  - Overfitting: may fit the noise in the data rather than the underlying trend and capture random fluctuations.

2/ Modeling With a Line

Using a Line to Predict

- Can we smooth out these binned means and close gaps? **A model.**
- Simplest possible way to relate two variables: a line.

$$y = mx + b$$

- Problem: for any line we draw, not all data is on the line.
    - Some data points will be above the line, some below.
    - $\rightsquigarrow$ Need a way to account for **chance variation** away from the line.

Linear Regression Model

- Model for the line of best fit:

$$Y_i = \underbrace{\alpha}_{\text{intercept}} + \underbrace{\beta}_{\text{slope}} \cdot X_i + \underbrace{\varepsilon_i}_{\text{error term}}$$

- **Coefficients/parameters** (α, β): true unknown intercept/slope of the line of best fit.

- **Chance error**, ε_i: accounts for the fact that the line doesn't perfectly fit the data.
  - Each observation allowed to be off the regression line.
  - The errors are 0 on average.

- Useful fiction: this model represents the **data generating process**.
  - George E.P. Box: "All models are wrong but some are useful."
  - not an exact depiction of reality

Interpreting the Regression Line

$$Y_i = \alpha + \beta \cdot X_i + \varepsilon_i$$

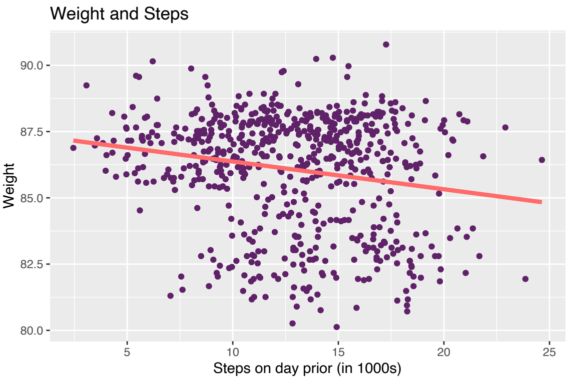
- **Intercept**, α: average value of Y when X is 0.
  - $\rightsquigarrow$ average weight when Jimmy takes 0 steps the day before.
- **Slope**, β: average change in Y when X increases by one unit.
  - $\rightsquigarrow$ average decrease in weight for each additional 1,000 steps.

Estimated Coefficients

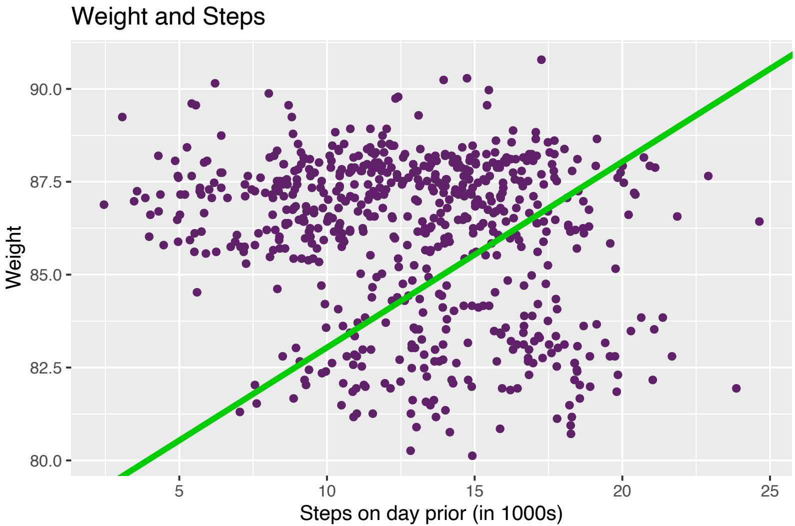
- Parameters: α, β
  - Unknown features of the **data generating process**.
  - Chance error makes these impossible to observe directly.
- Estimates: $\hat{\alpha}, \hat{\beta}$
  - An **estimate** is our best guess about some parameter.
- **Regression line**: $\widehat{Y} = \widehat{\alpha} + \widehat{\beta} \cdot x$
  - Average value of Y when X is equal to x.
  - Represents the best guess (or **predicted value**) of the outcome at x.

Line of Best Fit

```r
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple) +
    labs(
      x = "Steps on day prior (in 1000s)", y = "Weight",
      title = "Weight and Steps"
    ) +
    geom_smooth(method = "lm", se = FALSE, color = "indianred1", size = 1.5)
```



Q: Why Not This Line?



Prediction Error

- The key is to understand the prediction error for a line with intercept a and slope b.

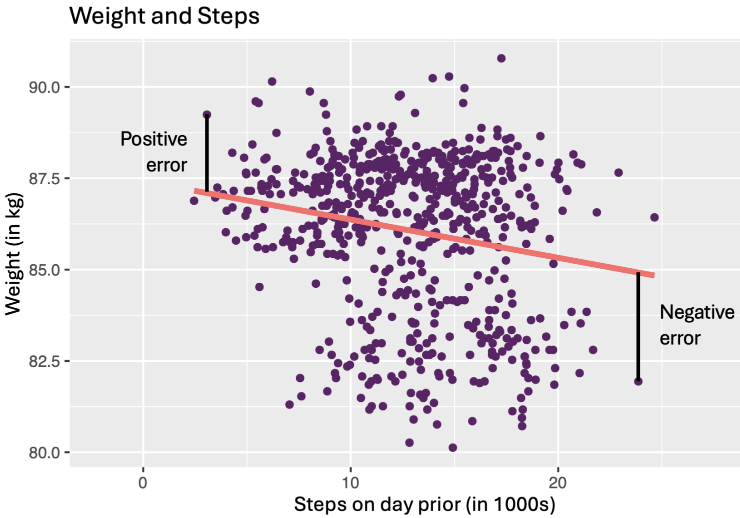
- **Fitted/predicted value** for unit i:

$$a + b \cdot X_i$$

- **Prediction error (residual)**:

$$\text{error} = \text{actual} - \text{predicted} = Y_i - (a + b \cdot X_i)$$

Prediction Error Visualized



Least Squares

- Get these estimates by the **least squares method**.
- Minimize the **sum of the squared residuals** (SSR):

$$\text{SSR} = \sum_{i=1}^{n} (\text{prediction error}_i)^2 = \sum_{i=1}^{n} (Y_i - a - b \cdot X_i)^2$$

- Finds the line that minimizes the magnitude of the prediction errors!

3/ Linear Regression in *R*

Linear Regression in R

- R will calculate/find least squares line for a data set using `lm()`.
  - Syntax: `lm(y ~ x, data = mydata)`
  - `y` is the name of the dependent variable
  - `x` is the name of the independent variable
  - `mydata` is the data.frame where they ‘live’!

```
> fit <- lm(weight ~ steps_lag, data = health); fit

Call:
lm(formula = weight ~ steps_lag, data = health)

Coefficients:
(Intercept)    steps_lag
    87.4168      -0.1047
```

Coefficients

- Use coef() to extract estimated coefficients:

```
> coef(fit)

(Intercept)   steps_lag
 87.4168013  -0.1046746
```

- **Interpretation**: a 1-unit increase in X (1,000 steps) is as sociated with a decrease in the average weight of 0.104 kilograms.

- **Q**: what would this model predict about the change in average weight for a 10,000 step increase in steps?

broom Package

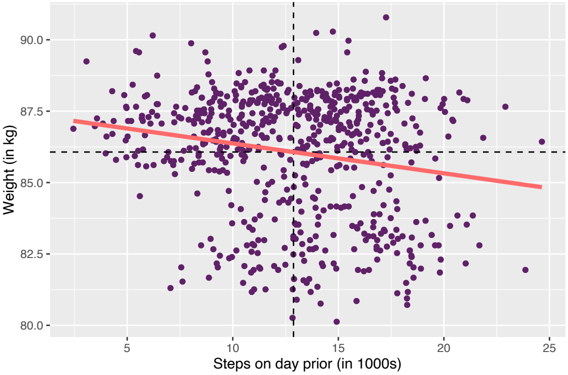
- The broom package can provide nice summaries of the regression output.
    - augment() can show fitted values, residuals and other unit-level statistics:

```
# Don't forget to install and load the package!
> require(broom)
> augment(fit) |> head()

# A tibble: 6 × 8
  weight steps_lag .fitted  .resid    .hat .sigma     .cooksd .std.resid
   <dbl>     <dbl>   <dbl>   <dbl>   <dbl>  <dbl>       <dbl>      <dbl>
1   86.7      17.5    85.6  1.12   0.00369   2.12 0.000513         0.526
2   86.2      18.4    85.5  0.714  0.00463   2.12 0.000264         0.337
3   85.8      19.6    85.4  0.474  0.00609   2.12 0.000154         0.224
4   86.3      10.4    86.3 -0.0340 0.00217   2.12 0.000000280    -0.0160
5   86.1      18.7    85.5  0.653  0.00496   2.12 0.000238         0.309
6   85.4       9.14   86.5 -1.03   0.00296   2.12 0.000349        -0.485
```

Properties of Least Squares Line

1. Least squares line always goes through $(\overline{X}, \overline{Y})$:

```
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple) +
    labs(x = "Steps on day prior (in 1000s)", y = "Weight (in kg)") +
    geom_hline(yintercept = mean(health$weight), linetype = "dashed") +
    geom_vline(xintercept = mean(health$steps_lag), linetype = "dashed") +
    geom_smooth(method = "lm", se = FALSE, color = "indianred1", size = 1.5)
```



Properties of Least Squares Line

2. Estimated slope is related to correlation:

$$\widehat{\beta} = (\text{correlation of } X \text{ and } Y) \times \frac{\text{SD of } Y}{\text{SD of } X}$$

- The division 'rescales' the unit-less correlation to the scale of Y.
- Thus, a unit change in $X \rightsquigarrow \widehat{\beta}$ change in Y

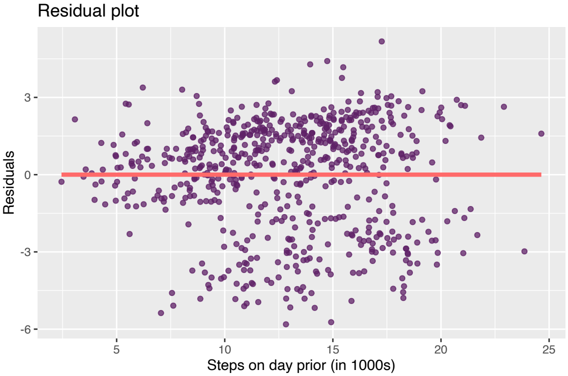
3. Mean of residuals is always 0! (or is it?)

```
> augment(fit) |>
    summarize(mean(.resid))

# A tibble: 1 × 1
  `mean(.resid)`
           <dbl>
1       2.14e-14
```

- Floating-point precision issues: computers represent numbers using a finite number of bits.
    - Leads to small rounding errors in computations.

Plotting the Residuals

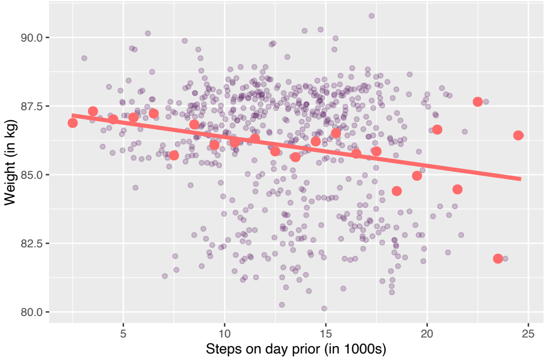
```
> augment(fit) |>
    ggplot(aes(x = steps_lag, y = .resid)) +
    geom_point(color = Tsinghua_Purple, alpha = 0.75) +
    labs(
      x = "Steps on day prior (in 1000s)", y = "Residuals",
      title = "Residual plot"
    ) +
    geom_smooth(method = "lm", se = FALSE, color = "indianred1", size = 1.5)
```



Smoothed Graph of Averages

- Another way to think of the regression line is a smoothed version of the binned means plot:

```
> ggplot(health, aes(x = steps_lag, y = weight)) +
    geom_point(color = Tsinghua_Purple, alpha = 0.25) +
    labs(x = "Steps on day prior (in 1000s)", y = "Weight (in kg)") +
    stat_summary_bin(fun = "mean", color = "indianred1", size = 3,
                     geom = "point", binwidth = 1) +
    geom_smooth(method = "lm", se = FALSE, color = "indianred1", size = 1.5)
```



4/ Model Fit

Beauty and Student Evaluations

- Why do some instructors at universities receive high teaching evaluations scores from students while others receive lower ones?
  - Data contains instructor and course information on 463 courses taught at UT Austin (`ModernDive`; see recommended resources in syllabus).

Variable	Description
`ID`	Identifier for courses taught at UT Austin
`score`	Avg. teaching score (1-5)
`bty_avg`	Instructor's avg. "beauty" score computed from a separate panel of six students (1-10).
`age`	Instructor's age

Table: UT Austin Teaching Eval. Data

- ⇝ Q: Which better predicts student evaluation scores: an instructor's perceived attractiveness (beauty score) or their age?

UT Austin Student Eval. Data

Import the data:

```
# Don't forget to load tidyverse and ggplot2!
> evals <- read_csv("https://bit.ly/46rSa7s"); evals

# A tibble: 463 × 14
      ID prof_ID score   age bty_avg gender ethnicity  language rank  pic_outfit pic_color cls_did_eval
   <dbl>   <dbl> <dbl> <dbl>   <dbl> <chr>  <chr>      <chr>    <chr> <chr>      <chr>            <dbl>
 1     1       1   4.7    36    5    female minority   english  tenu. not formal color               24
 2     2       1   4.1    36    5    female minority   english  tenu. not formal color               86
 3     3       1   3.9    36    5    female minority   english  tenu. not formal color               76
 4     4       1   4.8    36    5    female minority   english  tenu. not formal color               77
 5     5       2   4.6    59    3    male   not minor. english  tenu. not formal color               17
 6     6       2   4.3    59    3    male   not minor. english  tenu. not formal color               35
 7     7       2   2.8    59    3    male   not minor. english  tenu. not formal color               39
 8     8       3   4.1    51    3.33 male   not minor. english  tenu. not formal color               55
 9     9       3   3.4    51    3.33 male   not minor. english  tenu. not formal color              111
10    10       4   4.5    40    3.17 female not minor. english  tenu. not formal color               40
#  453 more rows
#  2 more variables: cls_students <dbl>, cls_level <chr>
#  Use `print(n = ...)` to see more rows
```

Fitting the Aesthetic Evaluation Model

```
> fit.bty <- lm(score ~ bty_avg, data = evals)
> fit.bty

Call:
lm(formula = score ~ bty_avg, data = evals)

Coefficients:
(Intercept)      bty_avg
    3.88034      0.06664
```

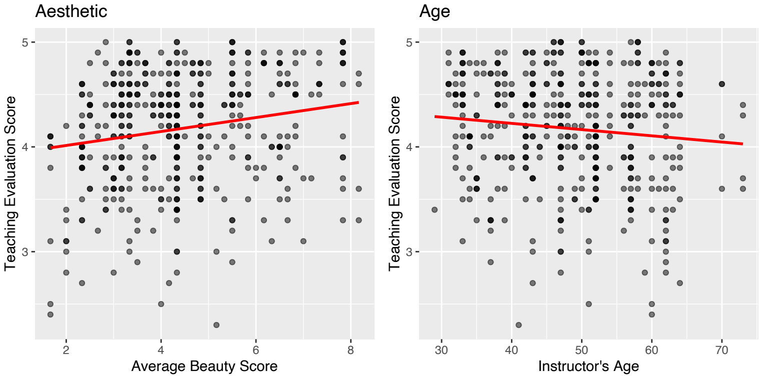
- Interpretation: For a one-point increase in average beauty score, the predicted evaluation score increases by approx. 0.066.

Fitting the Instructor Experience Model

```
> fit.age <- lm(score ˜ age, data = evals)
> fit.age

Call:
lm(formula = score ˜ age, data = evals)

Coefficients:
(Intercept)          age
   4.461932    -0.005938
```

- Interpretation: For a one-point increase in instructor's age, the predicted evaluation score decreases by approx. 0.005.

Comparing Models



- How well do the models “fit the data”?
  - How well does the model predict the outcome variable in the data?

Model Fit

- Model prediction error:

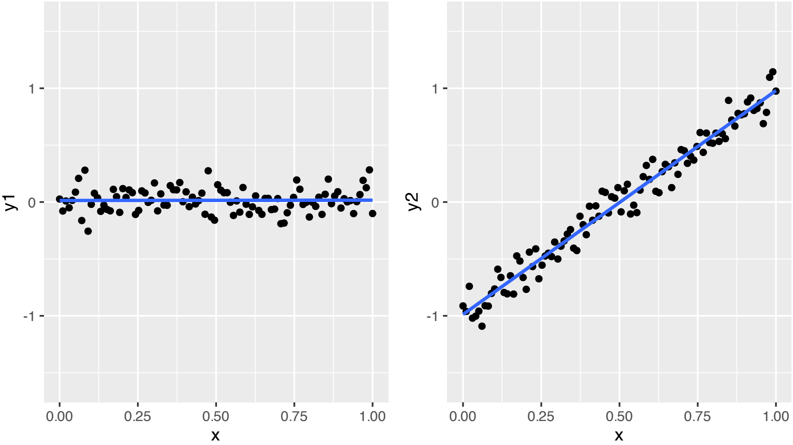
$$\text{prediction error} = \sum_{i=1}^{n} (\text{actual}_i - \text{predicted}_i)^2$$

- Prediction error for regression: **Sum of squared residuals**

$$\text{SSR} = \sum_{i=1}^{n} (Y_i - \widehat{Y}_i)^2$$

- Lower SSR is better, right?

These two regression lines have approximately the same SSR:



Benchmarking Model Fit

Benchmark our predictions using the **proportional reduction in error**:

$$\frac{\text{reduction in prediction error using model}}{\text{baseline prediction error}}$$

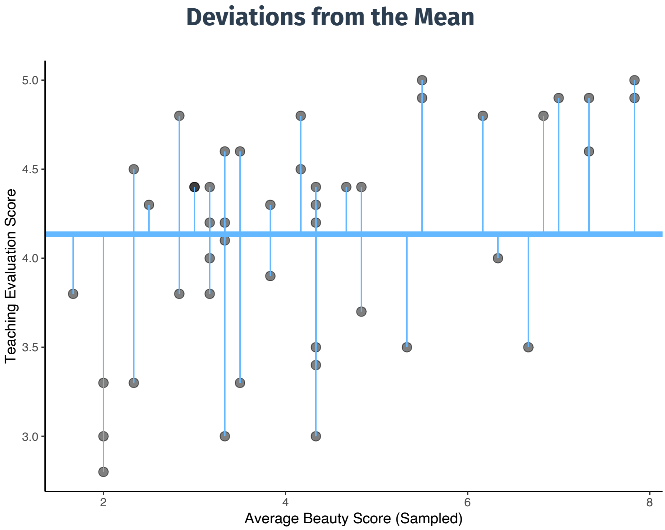
Baseline prediction error without a regression is using the mean of Y to predict. This is called the **Total sum of squares**:

$$TSS = \sum_{i=1}^{n} (Y_i - \overline{Y})^2$$

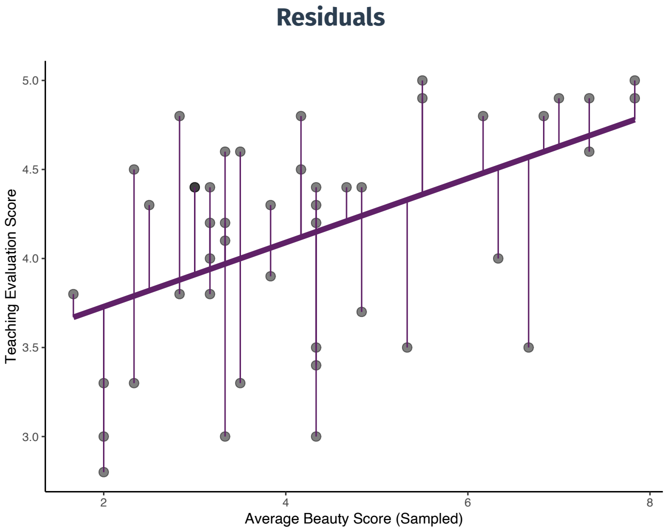
Leads to the **coefficient of determination**, R^2, one summary of LS model fit:

$$R^2 = \frac{TSS - SSR}{TSS} = \frac{\text{how much smaller LS prediction errors are vs. mean}}{\text{prediction error using the mean}}$$

Total SS vs. SSR



Total SS vs. SSR



Model Fit in *R*

- To access R^2 from the `lm()` output, use the `summary()` funciton:

```
> fit.bty.sum <- summary(fit.bty)
> fit.bty.sum$r.squared
[1] 0.03502226
```

- Compare to the fit using change in age:

```
> fit.age.sum <- summary(fit.age)
> fit.age.sum$r.squared
[1] 0.01145584
```

- Q: Which does a better job predicting teaching evaluation scores?

Accessing Model Fit via broom Package

- We can also access summary statistics like model fit using the `glance()` function from broom:

```
> library(broom)
> glance(fit.bty)
# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic    p.value    df logLik   AIC   BIC deviance
      <dbl>         <dbl> <dbl>     <dbl>      <dbl> <dbl>  <dbl> <dbl> <dbl>    <dbl>
1    0.0350        0.0329 0.535      16.7 0.0000508     1  -366.  738.  751.     132.
#  2 more variables: df.residual <int>, nobs <int>
```

  - `adj. r.squared`: 'penalizes' needlessly complex models (⇝ more on this later)
  - `sigma`: square root of the estimated variance of the residuals

- R^2 (or $adj.R^2$) is generally considered when we use linear regression.
  - We may also consider other model fit metrics, e.g., AIC (Akaike Information Criterion) or BIC
  - Lower AIC/BIC indicates better model fit

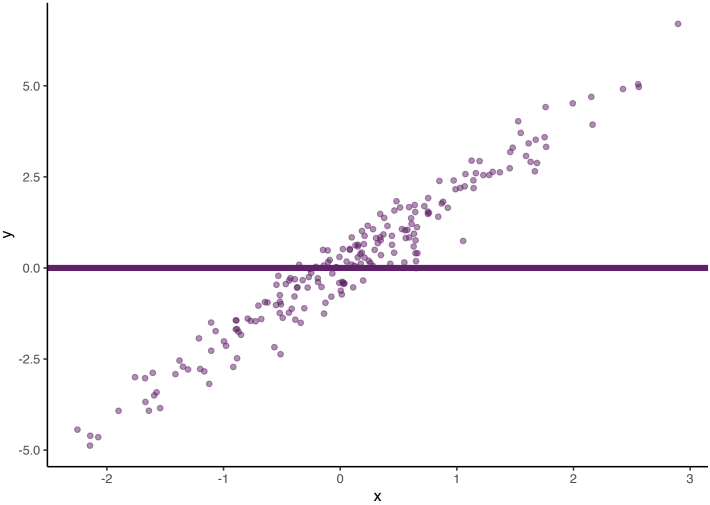
Simulated Data w/ Better Fit

- A bit difficult to see what's happening in the previous example.
- Let's look at simulated variables x and y:

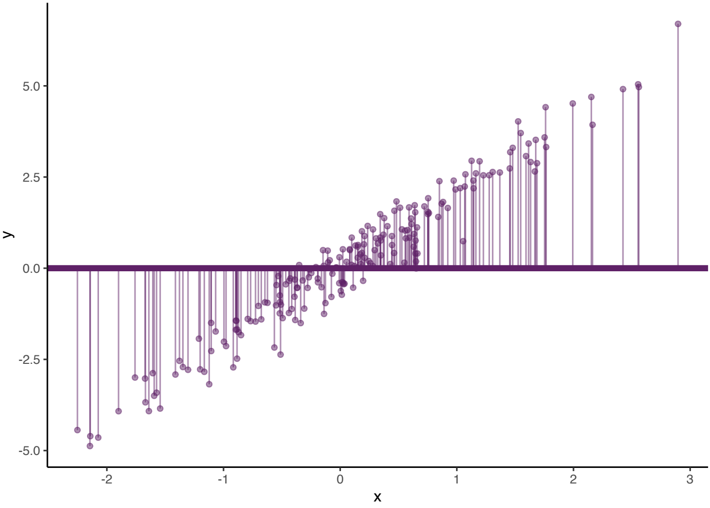
```
> fit.x <- lm(y ~ x)
```

- Very good model fit: $R^2 \approx 0.95$

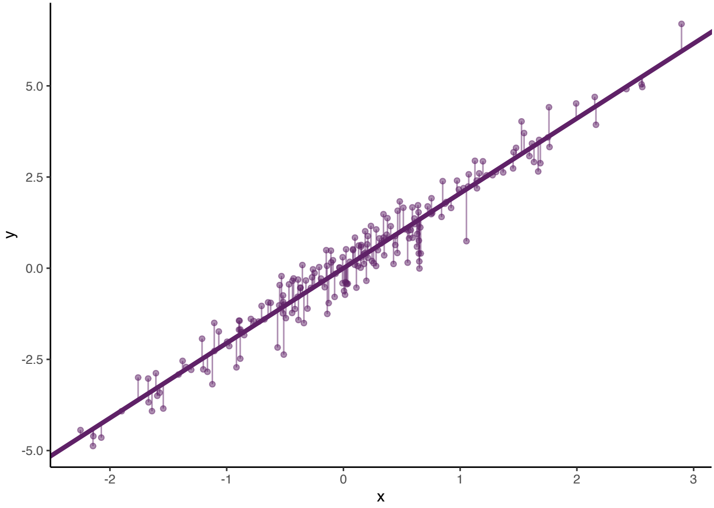
Simulated Data w/ Better Fit



Simulated Data w/ Better Fit

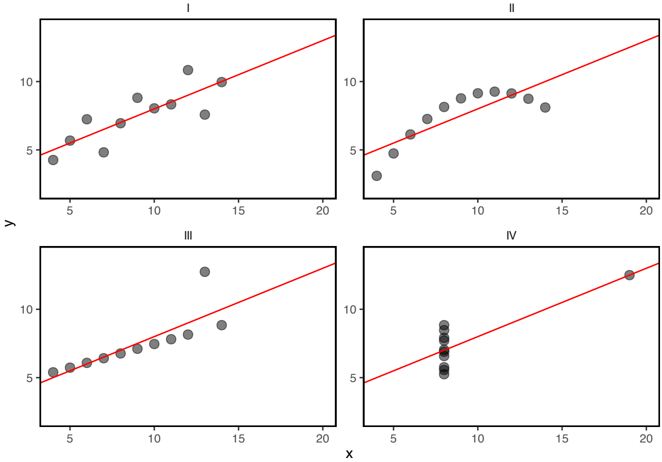


Simulated Data w/ Better Fit



Is R-Squared Useful?

- Can be very misleading. Each of these samples have the same R^2 even though they are vastly different:



Overfitting

- **In-sample fit:** how well your model predicts the data used to estimate it.
  - R^2 is a measure of in-sample fit.
- **Out-of-sample fit:** how well your model predicts new data.
- **Overfitting:** OLS optimizes in-sample fit; may do poorly out of sample.
  - Example 1: predicting airline profitability with oil price.
    - Until mid-2000s, oil price was a near-perfect predictor of airline profitability.
    - Decoupling of oil prices and airline profitability due to the mid-2000s oil crisis.
  - Example 2: predicting winner of Democratic presidential primary/nominee with gender of the candidate.
    - Until 2016, gender was a *perfect predictor* of who wins the primary.
    - Prediction for 2016 based on this: Bernie Sanders as Dem. nominee.
    - Q: Do you remember the result of the primary? (who ran in the general election in 2016?)
  - $\rightsquigarrow$ Bad out-of-sample prediction due to overfitting!

Enjoy Your Weekends! :)

Contact Information:
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/